Rails provides the class ActionController::Parameters, which wraps the hash and allows for the permit and require behavior needed to support strong parameters.

Other Security Resources

There's a limit to what you can test with security using TDD. It's a good idea to use a static analysis tool to look for security issues. Two options are Brakeman,[4] which you would run yourself, and CodeClimate, which automatically runs Brakeman on each commit.[5] Brakeman looks for a variety of security issues and provides some tips on working around them.

Prescription 35	Use an automatic security scanner to check for common security issues.

The Open Web Application Security Project has all kinds of useful information on security risks.[6] Of particular interest is WebGoat, a deliberately insecure application designed to allow you to hack and test solutions. The Rails version is called RailsGoat.[7]

What You've Done

In this chapter you applied testing techniques to security. You added a user model to the application and saw how to simulate a user login for test purposes. You then used that simulated login to test access control through the system.

You also used tests to isolate responsibilities and verify that certain users had access to only certain application resources. Then you applied tests to mass assignment and form modification, which required you to go outside normal integration tests. In the next chapter, you'll go beyond the application and interact with external services.

4. http://brakemanscanner.org
5. http://www.codeclimate.com
6. https://www.owasp.org
7. https://github.com/OWASP/railsgoat